

Segment-Dependent Chunk Length: Stages 1, 2, and 3

One Combined Report

Aryan Goyal

July 23, 2026

Abstract

An action-chunking policy predicts several future actions at once and then decides how many of them to execute before looking again. That number is usually a fixed constant. This project asks whether it should instead depend on the moment: commit a long chunk where the physics absorbs errors, and replan sooner where it amplifies them. This report combines the three stages of that investigation. Stage 1 tests the idea on short benchmark tasks, first with a perfect oracle signal and then with a learned predictor. Stage 2 tests it on long-horizon tasks that alternate between the two regimes, which is where the theory says the win should appear. Stage 3 proposes a constructed task where the regime mix becomes an experimental knob. Stage 1 is complete, Stage 2 is in its final evaluation, and Stage 3 is a design.

Contents

1	The hypothesis and the stage map	2
2	Stage 1a: does chunk length matter?	2
2.1	Setup	2
2.2	Results	3
2.3	Why the oracle only tied	3
3	Stage 1b: can the regime be read from frames?	3
3.1	Setup	3
3.2	Results	4
4	Stage 1c: does the predicted signal help in rollouts?	4
5	Stage 2: long-horizon alternating tasks	5
5.1	Setup	5
5.2	Checkpoint selection	5
5.3	The experiment, pending	6
5.4	Incidents worth recording	6
6	Stage 3: a constructed regime-switching task	6
7	Where things stand	7

1 The hypothesis and the stage map

Executing k actions without looking has two competing costs. If k is too long, the robot is blind through a moment where a small error grows into a failure. If k is too short, the policy queries itself constantly, and every query injects a fresh sample of the policy’s own error. The right k therefore depends on whether the current moment amplifies errors or absorbs them. We measure that property directly in simulation by perturbing one action and watching whether the trajectory deviation grows (open-loop unstable) or not (open-loop stable). The labeling procedure is documented separately in the labeling report.

Each stage answers one question.

Stage	Question	Signal source	Status
1a	Does chunk length matter, and in which direction?	perfect oracle	done
1b	Can a model read the regime from camera frames?	trained predictor	done
1c	Does the predicted signal help in rollouts?	trained predictor	done
2	Does it help on long alternating tasks?	predictor, zero-shot	running
3	Does the gain grow with more alternations?	task construction	proposed

The logic is sequential. If a perfect signal does not help (1a), no predictor can. If the regime cannot be read from frames (1b), the rule cannot leave the simulator. If the predicted signal helps in rollouts (1c), the remaining question is whether the benchmark tasks were ever the right test, which is Stage 2. Stage 3 exists because Stage 2 cannot control how much of each episode is stable transport and how much is unstable contact, and that mix is what decides how much room the switch has to win.

2 Stage 1a: does chunk length matter?

2.1 Setup

Two image-based diffusion policies trained on robomimic proficient-human data: square (epoch 950, success 0.90) and tool_hang (epoch 1200, success 0.90). Nothing about the policy changes between cells. Only the execution rule changes.

- **Fixed cells.** One chunk length for the whole episode, $k \in \{1, 2, 4, 8, 16\}$. Note that $k=1$ is ordinary per-step control.
- **Oracle cells.** Two chunk lengths $(k_{\text{free}}, k_{\text{contact}})$, switched by reading the simulator’s contact table at every query, with a two-query hysteresis. Grid: $(1, 4)$, $(1, 16)$, $(4, 1)$, $(8, 1)$, $(8, 4)$, $(16, 1)$, $(16, 4)$.

Contact is the oracle signal rather than the labels themselves because the true labels exist only on demonstration states, and a rollout leaves those states immediately. Contact is the best label-aligned observable available: the labeling report shows fixture contact above gripper contact above free space in mean divergence on every dataset. Each cell is 50 episodes, repeated at three environment seeds, so 150 episodes per cell and a standard error near 0.033.

2.2 Results

Cell	Square	Tool.hang
fixed $k=1$	0.793	0.667
fixed $k=2$	0.820	0.740
fixed $k=4$	0.807	0.793
fixed $k=8$	0.833	0.800
fixed $k=16$	0.827	0.807
oracle (8, 4)	0.873	0.807
oracle (16, 4)	0.793	0.780
oracle (1, 4)	0.773	0.780
oracle (4, 1)	0.793	0.600
oracle (8, 1)	0.820	0.667
oracle (16, 1)	0.780	0.633

Three findings, all on tool.hang, the contact-heavy task where the labels say the effect should appear.

1. **Short chunks hurt.** Fixed $k=1$ reaches 0.667 against 0.807 for $k=16$, a gap of 2.8 standard errors. Chunk lengths 4 through 16 form a plateau; the damage is concentrated at $k \leq 2$.
2. **Replanning every step during contact is the worst rule measured.** The three oracle cells that replan per step in contact reach 0.600, 0.633, and 0.667, which is 3 to 5 sigma below the plateau, in three independent cells. This is the direction the closed-loop labels predicted: the policy’s own corrections inject error, and contact amplifies it.
3. **The binary oracle never beats the best fixed k .** The best oracle cell (8, 4) reaches 0.807 and exactly ties fixed $k=16$.

Square stays flat across the whole grid (0.773 to 0.873), which is the expected null for a task whose policy is strong at every k .

2.3 Why the oracle only tied

The switching signal carried almost no information. Diagnostics show the oracle judged 84 to 88 percent of its tool.hang decisions to be in contact, so a binary switch is effectively constant and collapses to fixed k at its contact value. This also explains the cell-by-cell pattern: every cell with contact value 1 lands near fixed $k=1$, and every cell with contact value 4 lands near fixed $k=4$. What the labels actually contain is a continuous grade of instability, and binarizing it threw that away. Recovering it is the purpose of the predictor.

A secondary observation supports the injection mechanism: mean steps among successful episodes falls monotonically with commitment, from 167 to 145 on square and 480 to 436 on tool.hang. Per-step replanning dithers, commitment produces straighter trajectories.

3 Stage 1b: can the regime be read from frames?

3.1 Setup

A frozen V-JEPA 2 video encoder (ViT-L, 300M parameters, never updated) plus a small trained head (attentive pooling, proprioception fusion, two-layer MLP). Input per stamp is the last 16

camera frames and a 9-dimensional proprioception vector. Output is the probability the moment is unstable and a predicted divergence rate. Features are cached once, so head training is minutes per epoch. Training pool: about 155,000 labeled stamps across the four robomimic tasks, two MimicGen sets, ten LIBERO-Long files, and policy rollout states. Splits are by demonstration, never by stamp, because stamps 2 steps apart are nearly identical.

3.2 Results

Run	AUROC all	AUROC confident	Note
1: pooled head	0.699	0.868	the head used downstream
2: square only	0.768	0.719	single-task training
2: square to tool_hang	0.562	0.465	transfer test
3b: pooled plus LIBERO	0.720	0.893	pinned validation
4: DINOv2, single frame	0.703	0.859	motion ablation
5: closed-loop labels	0.627	n/a	diagnostic head

“Confident” restricts scoring to stamps outside the deadband, which are the only stamps whose binary label is proven rather than ambiguous. Four conclusions.

1. **The pooled head works where the labels are confident.** 0.868 means the head separates proven-unstable from proven-stable moments from frames plus proprioception. The lower all-stamps number is expected, since deadband stamps are ambiguous in the ground truth itself.
2. **Single-task training does not transfer.** Square to tool_hang lands at chance or below. A head trained on one task learns task-specific visual cues, not stability. Every task family we care about must contribute labeled data to the pool.
3. **Motion context adds almost nothing.** A single-frame DINOv2 head matches the 16-frame video head to within 0.009. The signal the head reads is scene configuration at the query moment, not motion. A cheaper single-frame encoder would serve the switch nearly as well.
4. **Closed-loop expansion is only weakly predictable.** 0.627, well above chance but nowhere near deployable. This is consistent with the mechanism: closed-loop expansion depends on how the policy reacts, which is not fully visible in the scene. Nothing downstream uses this head.

One methodological note worth carrying forward. The first LIBERO comparison drew its train and validation split randomly over the combined pool, which leaked most of the baseline’s validation demonstrations into training and showed a fake improvement of 0.968. Pinning the validation demonstrations showed the true gain, 0.018. Every pool-variant comparison now pins the split.

4 Stage 1c: does the predicted signal help in rollouts?

The frozen pooled head goes into the control loop. At every replan the last 16 frames and current proprioception pass through the encoder and head, and the predicted divergence rate picks the chunk length: 16 if at or below the training threshold 0.0355, and 4 if above it. Inference runs in a separate process, about 100 ms per replan. Three seeds pooled, 150 episodes per cell.

	Predictor (16, 4)	Best fixed k	Best oracle cell	Fixed $k=1$
square	0.767	0.833	0.873	0.793
tool_hang	0.800	0.807	0.807	0.667

On tool_hang the predictor statistically ties both the best fixed k and the best oracle cell, while avoiding the 0.60 to 0.67 trap of replanning through contact. Its diagnostics show it is not merely reproducing the contact signal: it calls unstable on 71.6 percent of decisions against the oracle’s 84 to 88 percent contact rate, and its mean executed chunk is 11.4. It discriminates within contact, committing through contact stretches it judges benign, which is the continuous-signal advantage Stage 1a said a binary oracle could not express. On square the predictor sits at the bottom of the flat band, which is the expected null on a ceiling task.

What Stage 1 did not show is the switch beating the best fixed k . Both tasks are short and 65 to 88 percent contact, so a single fixed k is already near optimal and a switch has little room. A tie is close to the ceiling available on these tasks. Testing the idea properly requires tasks that spend substantial time in both regimes, which is Stage 2.

5 Stage 2: long-horizon alternating tasks

5.1 Setup

Four MimicGen tasks whose episodes are long and alternate between free-space transport and contact-rich stages: three_piece_assembly, nut_assembly, kitchen, and coffee_preparation, all D0, 1,000 demonstrations each, mean episode lengths 337 to 689 steps. Each trains an image-based diffusion policy for 2,000 epochs, which is 200,000 gradient steps.

One engineering constraint shapes everything. MimicGen environments require robosuite 1.4, and our diffusion policy stack requires robosuite 1.5, and the two cannot coexist in one process. The solution is a strict split: dataset conversion and environment ownership in one virtual environment, policy training and inference in another, with a bridge process that streams observations and actions between them over a pipe. All evaluation is therefore post hoc rather than during training.

The predictor head is applied zero-shot, with no relabeling of these tasks. A distribution check on MimicGen frames confirmed the head is not degenerate there: predicted divergence spans the threshold on all four tasks, with 11 to 55 percent of stamps above it, and the ordering is plausible, since coffee_preparation reads most unstable and kitchen most stable. Relabeling remains the fallback if the zero-shot switch underperforms.

5.2 Checkpoint selection

All four trainings finished at epoch 2,000. Selection evaluates five checkpoints per task at 25 episodes each, seed 0, with horizons set per task from the demonstration lengths (560, 650, 980, and 1,140 steps). The rule is highest success rate, ties broken toward the later epoch.

	ep. 600	ep. 1000	ep. 1400	ep. 1700	ep. 2000
kitchen	0.96	1.00	1.00	1.00	1.00
three_piece_assembly	0.48	0.52	0.64	0.80	0.68
nut_assembly	0.12	0.00	0.32	0.32	0.36
coffee_preparation	0.04	running	running	running	pending

Kitchen is at ceiling, three_piece_assembly is strong, and nut_assembly is weak but usable. Coffee_preparation is the weakest: partial counts from the running cells put it near 0.15 to 0.25 rather than climbing, so its cells will carry wide error bars and it may not carry a comparison on its own. The four-task design anticipated one weak task.

5.3 The experiment, pending

On the selected checkpoints, fixed $k \in \{1, 4, 8, 16\}$ plus the predictor switch (16, 4), 50 episodes per cell at seed 0, same policy weights across all cells. Twenty cells across eight GPUs, roughly a day of compute. The runner is built and smoke-tested end to end, including the three-process path through environment, policy, and predictor. It waits on the last selection cells.

The prediction under test is specific: on these tasks the switch should beat every fixed k , because alternating structure forces any fixed choice to pay one of the two costs at some point in every episode.

5.4 Incidents worth recording

Two training runs were lost and rebuilt. The first loss looked like a random disk event; the second exposed the cause. This virtual machine reprovisions its scratch disk on every machine stop, so both losses were the machine being stopped, not a fault. A separate loss came from a relaunch script that answered an overwrite prompt automatically and deleted the surviving checkpoint grids. The process changes are in place: the launcher refuses to start when an experiment directory exists, checkpoint grids are copied before any evaluation touches them, and power-off is left to a watchdog that fires only when trainings, selection, and the experiment have all reported completion. Net cost was about two days.

6 Stage 3: a constructed regime-switching task

Stage 2 tests the switch on natural long-horizon tasks, but it cannot control how much of each episode is stable transport and how much is unstable contact, and that mix decides how much room the switch has. Stage 3 removes the limitation by constructing the task: a repeated transport-and-insert task in robosuite with N pegs spaced far apart, so each cycle is a long free-space transport followed by a tight insertion, repeated N times. The two quantities the theory cares about become knobs, namely the number of alternations N (swept over 1, 2, 4) and the ratio of stable to unstable segment length (set by spacing and tolerance). The prediction is that the gap between the switched policy and the best fixed k grows with N , because every added alternation forces a fixed k into one more compromise. The comparison is the same policy at its best fixed k , found by a full sweep, against oracle switching and then predictor switching. Because we define the task, regime boundaries are known by construction, which gives a free and perfect oracle, a ground-truth check of the perturbation labeler against the designed boundaries, and clean predictor training labels. Demonstrations come from MimicGen: author the task, record about ten source demonstrations, and generate one thousand. The known weakness is that a bespoke task invites the criticism that it was engineered to win, so Stage 3 is positioned as the mechanism experiment that explains and quantifies the effect while Stage 2 remains the naturalistic evidence. It starts after the Stage 2 results are in, since those results decide whether it quantifies how the win scales or diagnoses which regime mix the hypothesis needs.

7 Where things stand

What is established. Chunk length matters, and it matters in the direction opposite to the naive control-theory prescription: replanning fastest where the system is least stable is the worst rule measured, at 3 to 5 sigma across independent cells. The mechanism is confirmed at two levels, since the closed-loop labels show the policy’s own corrections inject error at 90 to 99 percent of measured states, and the rollouts show per-step replanning losing 0.14 to 0.21 success on the contact-heavy task. The regime is readable from vision at AUROC 0.868 on confident stamps, and the resulting switch runs in a real control loop at about 100 ms per decision.

What is not established. The switch has not yet beaten the best fixed chunk length. On the Stage 1 tasks it ties, and the diagnostics explain why: those tasks are short and mostly contact, so a fixed k is already near optimal and a tie is close to the available ceiling. That is consistent with the hypothesis but is not evidence for it.

What decides it. The Stage 2 experiment, on tasks chosen so that no fixed k can be right throughout. Its preconditions are now met, since three of four base policies are strong enough to make the comparison meaningful and the predictor is not degenerate on these scenes. If the switch wins there, the claim is demonstrated on naturalistic long-horizon tasks and Stage 3 quantifies how the win scales with the number of alternations. If it ties again, that is a genuine negative result, and Stage 3 becomes the diagnostic that isolates which regime mix the hypothesis actually needs.